



Events and forms



Learning Objectives

- Explain what a browser **event** is and give examples of common ones
- Explain how to pass a **callback** to **addEventListener()** with the argument **event**
- Use JavaScript to listen to an **event** and execute expected behaviour when the **event** is dispatched
- Prevent an element's default behaviour using **preventDefault()**



Events

So you've now seen **how** to update the page using JS. In this section, we're going to be looking at how JS and the DOM can work together to program **when** something should happen

This is a **super important** step towards building web applications, as we want to be able to **react** when something happens on our page. Not always, but **most of the time**, that something will be a user **performing some action**

Q: Can you think of some actions the user might perform on our page?

Note: There's a TON of events made available to us. For a full list, you can check out [MDN's Event page](#)



Events

Ok, so events are always happening, but how do we actually react to them? To do so, we first have to be **listening** to the event we care about

Every element gives us access to a method called **addEventListener**. This method takes in 2 main arguments*:

- **A string**: The name of the event we want to listen to
- **A function**: The action we want to perform when the event happens

```
document.body.addEventListener('click', function() {  
  console.log('How dare you click me!?!')  
})
```

Q: What will this code do?

A: Let's try it out!

**There's an optional third argument that allows you to configure the event listener, as always, MDN is your friend if you need to know more or you're feeling curious. Terms and conditions apply. :)*



Events

Again, you can choose to use an **anonymous function** inline:

```
document.body.addEventListener('click', function() {  
  console.log('What a wonderful world!')  
})
```

Or use a **declared function** and then pass it to addEventListener:

```
function greet () {  
  console.log('What a wonderful world!')  
}  
  
document.body.addEventListener('click', greet)
```

These two snippets do **exactly the same thing** – let's try both out!



Time for some live coding

Let's build a simple counter app together!

We need 3 things:

- **A display for our counter**
- **A way to make it go up**
- **A way to make it go down**

Extended

- **Add a way to reset it back to 0**
- **Make the counter go up on its own every second**
- **Display the word "second(s)" next to it**



Time for some practice

- Create a paragraph element on the page
- Add an event listener to the element, targeting the 'click' event.
- Every time you click it, change the element background colour to a random colour!



Updating the DOM with user input

Q: Can you think of what element and event type we might be able to use?



Forms

Remember **forms**? Up until now, we've used forms but they weren't really **doing anything yet**. Now, thanks to JS, we can use them to give the user a **proper way** to **give us information** and react to it!



Forms

```
// somewhere in your index.html
```

```
<input class="favourite-food" placeholder="What's your favourite food?" />
```

```
// index.js
```

```
const foodInput = document.querySelector('.favourite-food')
```

```
console.log(foodInput.value)
```

Q: What do you think this will do?

A: Let's try it out!



Forms

```
// somewhere in your index.html  
<input class="favourite-food" placeholder="What's your favourite food?" />
```

```
// index.js  
const foodInput = document.querySelector('.favourite-food')  
foodInput.addEventListener('input', function() {  
  console.log(foodInput.value)  
})
```

Q: And this?

A: Let's try it out!



Forms

```
// somewhere in your index.html

<input class="favourite-food" placeholder="What's your favourite food?" />

// index.js

const foodInput = document.querySelector('.favourite-food')

// note - our function now has an argument!
foodInput.addEventListener('input', function(event) {
  console.log(event.target.value)
})
```

Q: And what about this one? 🤔

A: Let's try it out!



Interlude: Callbacks

When you pass a function as an argument to another function (a **callback function**), you can then invoke the **callback** with **arguments**.

In the case of **addEventListener**, when it invokes its **callback**, the first **argument** will always be the **event** object.

```
document.body.addEventListener('click', function(event) {  
  console.log(event)  
})
```

NB: To have access to the event object, **declare it as a parameter** in your **callback** function!

Events give us a TON of useful info! Let's explore it in the browser!

Note: Always check MDN if you want to know what arguments your function will receive. I.e.



Forms

That's right, the first snippet with the lone `console.log` only logs what the value is **at that time**, and **never again**

The second snippet one creates a **listener** that will **log** the value **any time it changes**

In the third one, the **event** has a **target** property which points to the **element** that triggered the event, so the same result is produced since both **foodInput** and **event.target** refer to the **same element**



Forms

When a user **submits** a form, the form element triggers a **submit event**. We can listen to that event whenever we want to **do something** with the submission in JS

```
// somewhere in your index.html
<form class='favourite-food-form'>
  <input class='favourite-food' placeholder="What's your favourite food?" />
  <button type='submit'>SEND</button>
</form>
```

```
// index.js
const foodForm = document.querySelector('.favourite-food-form')
foodForm.addEventListener('submit', function (event) {
  // whatever we want to do here...
})
```

Q: What will happen right now when you submit this form?



Forms

```
const foodForm = document.querySelector('.favourite-food-form')
foodForm.addEventListener('submit', function (event) {
  event.preventDefault()
  // do your thing here...
})
```

The page **refreshes** every time we submit! That's annoying, since the whole point of using JavaScript is that we can do things in **real time**, without having to refresh the page, but the form is stuck in its old ways...

To fix that, we can use another useful method provided by the event: **.preventDefault()**

This is the equivalent of us telling the browser:

“Look, I know your default behaviour here is to refresh the page, but let me take over, I know what I’m doing!”



Forms

```
// index.js
const foodForm = document.querySelector('.favourite-food-form')
foodForm.addEventListener('submit', function (event) {
  event.preventDefault()
  const foodInput = foodForm.querySelector('.favourite-food')
  console.log(foodInput.value)
})
```

After preventing the page from refreshing, we can go on ahead and do our work

Interesting to note here how we can actually use `querySelector` on **elements other than document**

Q: What's the difference between `document.querySelector()` and `foodForm.querySelector()`?



Forms

Great work! Hopefully you are starting to see how we can **combine** these **inputs** and **listeners** to create entire applications that **dynamically render** what the user needs on the page

Let's explore with a little practice...



Time for some practice

- **Create a form with an input**
- **Whenever the user submits the form, add whatever they've entered to a list below the form**

Extended

- **Only create the new item if they've entered more than 3 characters**
- **Clear the input once the item is added**